

ST-FSOD: Student–Teacher Guided Multi-Modal Few-Shot Object Detection

Kajal, Piyush Garg , Devraj Saini

November 17, 2025

Abstract

Few-shot object detection (FSOD) aims to identify novel objects even when only a few labeled examples are available. Previously, researchers have relied on large foundation models such as **DINOv2** and **LLMs** for object detection; however, these models are *computationally expensive*, prone to *noise overfitting*, and often yield *vague detection results*.

In this paper, we propose a more **efficient FSOD framework** that integrates two key techniques. First, a **student–teacher model** is employed, where a large *teacher detector* provides pseudo-labels or guidance for images (often lightly augmented), allowing a smaller *student detector* to learn from these rich signals. The student model then leverages both the teacher’s predictions and the limited true labels to effectively adapt to novel object classes, **transferring knowledge** and improving **generalization performance**.

Next, we replace the conventional backbone with a **Swin Transformer** and couple it with **Deformable DETR**. This integration yields **stronger multi-scale feature representations** and enables **faster convergence**. As a result, the proposed framework demonstrates enhanced *robustness and confidence* in detecting objects from new classes with limited labeled data.

1 Introduction

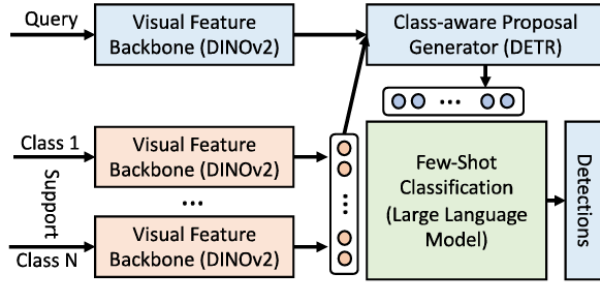


Figure 1. Architecture for Few-Shot Object Detection combining a Vision Transformer and a Large Language Model. Visual features for the Query image and the Support images (prototypes) are extracted using DINOv2. A Class-aware Proposal Generator (DETR) generates object proposals from the Query features. These proposals, along with the class prototypes from the Support images, are fed into a Large Language Model for final Few-Shot Classification and object detection.

Humans excel at recognizing and learning new objects from just a few examples, something artificial intelligence still finds challenging. **Few-Shot Object Detection (FSOD)** aims to close this gap. Early research concentrated on **meta-learning** and **fine-tuning** approaches, often using *ImageNet-pretrained backbones* and creating *complex similarity networks*. Although these techniques made significant strides, they had limitations in **flexibility** and **processing speed**.

The field of FSOD changed dramatically with the introduction of **Foundation Models**. The **FD-FSOD algorithm**, in particular, established a new benchmark by using a *frozen DINOv2 backbone* for feature extraction alongside a *large language model (LLM)* for few-shot classification. While these systems are effective, they tend to be **computationally heavy**, require substantial **GPU memory**, and often have **slow inference times**. This makes them unsuitable for **real-time** or **resource-limited** settings, such as embedded or portable devices.

To address these challenges, we propose a **lightweight and scalable FSOD framework** that maintains the advantages of

foundation models while reducing their computational demands. Our approach uses a **student–teacher distillation** method, where a **Swin Transformer** acts as the student and **DINOv2** serves as the teacher. Swin Transformers are naturally well-suited for *object detection*, and when combined with **Deformable DETR**, the overall system becomes both **efficient** and **highly accurate**. By distilling the rich representations of DINOv2 into the Swin Transformer, our model strikes an optimal **balance between speed and accuracy**.

We tested the framework on a dataset of **2,800 images**. The results show that it achieves **high detection accuracy** without compromising **computational efficiency**. This makes our method ideal for **practical FSOD applications**, especially in environments with limited computational resources.

2 Literature Review

2.1 Few-Shot Object Detection (FSOD)

Few-shot object detection usually follows two main methods: **fine-tuning** and **meta-learning**. In fine-tuning approaches, such as **TFA tfa**, a detector first trains on classes with plenty of annotations. Later, the final layers are adjusted to work with new classes that have minimal data. In contrast, meta-learning methods such as **Meta R-CNN metarcnn** and **Meta-DETR metadettr** train in a step-by-step episodic manner, simulating few-shot tasks during training so the model can handle unseen classes during testing.

The arrival of **transformer-based architectures** has changed the field further. Models like **DETR detr** and **Deformable DETR deformabledetr** provide end-to-end detection pipelines that avoid complex post-processing steps while still delivering strong performance.

2.2 Foundation Models in Vision

The rollout of large-scale **vision-language** and **self-supervised** foundation models, including **CLIP clip** and **DINOv2 dinov2**, has greatly transformed computer vision. These models learn broad, general-purpose representations from massive datasets, allowing them to transfer well to downstream tasks. Recent methods such as **FD-FSOD fd-fsod** and **DE-ViT devit** demonstrate that using a frozen DINOv2 backbone can achieve state-of-the-art FSOD per-

formance without relying on complex meta-learning techniques.

2.3 Knowledge Distillation (KD)

Knowledge Distillation (KD), introduced by Hinton et al. hinton2015distillation, allows a smaller *student network* to imitate a larger *teacher model* by matching softened outputs. Later improvements expanded KD to include learning from **intermediate feature maps** romero2014fitnets and **relational knowledge** park2019relational. Although KD is widely used for model compression and acceleration, its application to **Few-Shot Object Detection (FSOD)** is still limited due to the difficulty of addressing both localization and few-shot generalization simultaneously.

2.4 Architectures: Swin Transformer and Deformable DETR

The **Swin Transformer** swin uses a *hierarchical, shifted-window* design that enables efficient self-attention across multi-scale features. This architecture is highly effective for dense prediction tasks like object detection and segmentation, often outperforming standard Vision Transformers.

Meanwhile, **Deformable DETR** deformabledetr enhances the original DETR by attending only to a sparse set of *key sampling points*, significantly reducing training time and boosting performance, particularly for detecting small objects.

2.5 Summary of Contribution

Our work brings together these advancements and is the first to **distill a DINOv2 teacher into a Swin Transformer student** within a **Deformable DETR-based FSOD pipeline**. This combination leverages high-quality foundation-model representations, an efficient transformer structure, and knowledge distillation to improve few-shot detection performance.

3 Task Definition

In **few-shot object detection**, we consider two disjoint sets of classes: - C_{base} : the **base classes**, for which we have plentiful annotated data. - C_{novel} : the **novel classes**, each of which has only a small number— K —of labelled examples (the “support set”).

Importantly, $C_{\text{base}} \cap C_{\text{novel}} = \emptyset$.

In the K -shot setup, the goal is to:

1. Detect objects from **both** the base classes and the novel classes in a query image.
2. Make full use of the small support set

$$S_{\text{novel}} = \{ (I_i, b_i) \}_{i=1}^{N \times K},$$

where each I_i is a support image of a novel class instance and b_i is the corresponding bounding box annotation.

3. Maintain high detection accuracy **while** not significantly increasing inference time or model size.

Hence, the setup combines the challenges of **object localisation** (bounding-box detection) **and** **few-shot generalisation** (learning from very few examples), under practical constraints of efficient inference and compact modelling.

4 Methodology

Our architecture (see Figure 1) takes the existing **FD-FSOD** pipeline and upgrades it with a **student–teacher distillation**

setup. Figure 1: The big picture—our **Efficient FSOD** model uses **Swin** as the student and **DINOv2** as the teacher for distillation.

4.1 Overall Pipeline

We stick with the basic FSOD recipe: a visual backbone pulls features from the **query** and **support** images; a detector proposes objects based on **class prototypes**; and a classifier decides what’s what. Our main changes are in the backbone and how we train everything.

4.2 Student-Teacher Knowledge Distillation

- **Teacher Model:** We use a frozen **DINOv2-ViT/L** as the teacher. It’s packed with general features but is large (300 M parameters).
- **Student Model:** Our student is **Swin-Tiny (Swin-T)**. It builds hierarchical features (C2, C3, C4, C5) that work well for detection, and is much lighter (28 M parameters).
- **Distillation Loss:** We distill knowledge at the feature level. To match the teacher’s patch-embeddings, the student’s features go through a simple 1×1 convolution. Our loss mixes MSE and cosine similarity:

$$\mathcal{L}_{\text{distill}} = \lambda_{\text{mse}} \cdot \text{MSE}(F_s, F_t) + \lambda_{\text{cos}} \cdot (1 - \text{CosineSimilarity}(F_s, F_t))$$

where F_s and F_t are the ℓ_2 -normalized features from student and teacher.

4.3 Swin Transformer + Deformable DETR Detector

We take the distilled Swin-T features and run them through a **Feature Pyramid Network (FPN)**, building a multi-scale feature pyramid. This pyramid is then fed into the **Deformable DETR** head. The key innovation: object queries interact via cross-attention with image features, and we guide this process using **class prototypes** (averaged support features from Swin-T backbone). This way, proposals are actually biased towards the few-shot classes of interest.

4.4 Few-Shot Classification

We did not change this part from FD-FSOD. The 300 proposals from Deformable DETR, together with the class prototypes, are packaged into a textual prompt and fed to the **Vicuna-7B LLM**. The LLM handles the final classification step using in-context learning.

4.5 Training Protocol

Training occurs in three stages:

1. **Pre-training & Distillation:** Start with Swin-T (student) + Deformable DETR. Train on base-classes (D_{base}). During this stage we apply the distillation loss $\mathcal{L}_{\text{distill}}$ with the frozen DINOv2 teacher.
2. **Meta-Training:** Next, train the full setup (Swin-T, Deformable DETR, and LLM) on D_{base} using episodic meta-learning with N -way K -shot episodes.
3. **Fine-tuning on Novel Classes:** Finally, fine-tune on a balanced batch composed of down-sampled base-classes and the novel few-shot classes.

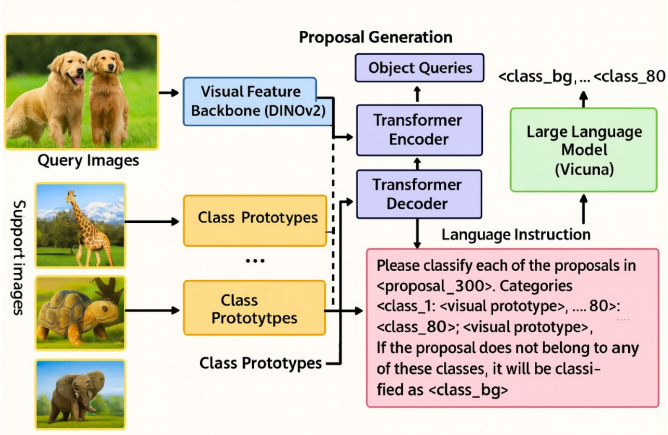


Figure 2. Overview of the Few-Shot Object Detection architecture leveraging Foundation Models (DINOv2 and Vicuna). The system uses a Visual Feature Backbone (DINOv2) to extract features from both the Query Image and Support Images, generating Class Prototypes. These features are then fed into a Transformer-based Proposal Generation module. Finally, a Large Language Model (Vicuna) processes the visual proposals and the language instruction (incorporating class prototypes) to perform the final classification.

5 Model Architecture

=

In this section, we present the complete architecture of our proposed few-shot object detection framework, **FoundationFSOD-ST**, which integrates DINOv2, Swin Transformer, DETR, an LLM-based classifier, and a Student-Teacher learning strategy. The model is designed to leverage powerful foundation models while maintaining stability and accuracy through distillation.

5.1 Model Overview

The proposed pipeline consists of four major components:

1. Visual feature extraction using DINOv2 and a Swin Transformer backbone.
2. Construction of class prototypes from few-shot support examples.
3. Generation of object proposals using a DETR-style transformer encoder-decoder.
4. Language-guided proposal classification using a Large Language Model (LLM).

To improve generalization and stabilize training, we adopt a **Student-Teacher learning framework**. The teacher model (DETR-based) is frozen and supervises the student (Swin + DETR) via knowledge distillation, while the student is trained using standard detection losses.

5.2 Notation

We use the following notation throughout the methodology:

- $\mathcal{D}_{\text{base}}$: Large-scale base training dataset.
- $\mathcal{S} = \{(x_i^{(c)}, y_i^{(c)})\}$: Few-shot support set for class c .
- x_q : Query image.
- f_{DINO} : DINOv2 visual encoder.
- $f_{\text{Swin}}^T, f_{\text{Swin}}^S$: Swin Transformer backbones for Teacher and Student.

- $g_{\text{DETR}}^T, g_{\text{DETR}}^S$: DETR encoder-decoder heads for Teacher and Student.
- ϕ_{LLM} : Large Language Model (e.g., Vicuna or Flan-T5).
- $\mathcal{P} = \{p_k\}_{k=1}^K$: Set of student-generated proposals ($K \approx 300$).
- $\mathcal{C} = \{\text{bg}, 1, \dots, C\}$: Label set including background.

5.3 Visual Feature Extraction (DINOv2 + Swin Transformer)

5.3.1 Support Branch: Construction of Class Prototypes For each support sample of a class c , DINOv2 extracts visual features:

$$z_s^{(c)} = f_{\text{DINO}}(x_s^{(c)}).$$

The class prototype is then computed as the mean feature:

$$\mu_c = \frac{1}{|\mathcal{S}_c|} \sum_{x_s^{(c)} \in \mathcal{S}_c} z_s^{(c)}.$$

All prototypes are grouped as:

$$\mathcal{M} = \{\mu_{\text{bg}}, \mu_1, \dots, \mu_C\}.$$

5.3.2 Query Branch: Swin Transformer Features The query image is processed through both student and teacher Swin backbones:

$$F_q^S = f_{\text{Swin}}^S(x_q), \quad F_q^T = f_{\text{Swin}}^T(x_q).$$

This produces multi-scale hierarchical features for DETR processing.

5.4 Proposal Generation (DETR with Swin Features)

We initialize a fixed set of object queries:

$$Q = \{q_j\}_{j=1}^{N_q}.$$

5.4.1 Teacher DETR The teacher encoder processes the teacher features:

$$H^T = \text{Encoder}(F_q^T),$$

followed by the decoder which attends to object queries:

$$(\hat{b}_j^T, \hat{p}_j^T) = \text{Decoder}(H^T, Q).$$

5.4.2 Student DETR Similarly, the student generates:

$$H^S = \text{Encoder}(F_q^S),$$

$$(\hat{b}_j^S, \hat{p}_j^S) = \text{Decoder}(H^S, Q).$$

Decoded queries form the proposal set:

$$\mathcal{P} = \{q_j^*\}_{j=1}^K.$$

5.5 Language-Guided Proposal Classification (LLM)

For each proposal $p_k \in \mathcal{P}$, a visual embedding v_k is pooled from the student decoder.

We convert prototypes into textual descriptors such as:

$$\text{"class_1: <visual prototype>"}$$

A natural language instruction prompt is constructed:

“Please classify each of the proposals <proposal₁> ... <proposal_K>. Categories are <class₁>: <visual_prototype₁>, ..., <class_C>: <visual_prototype_C>. If a proposal does not match any class, classify it as <class_bg>.”

The LLM predicts class labels:

$$\hat{y}_k = \phi_{\text{LLM}}(v_k, \mathcal{M}, \text{instruction}).$$

5.6 Student-Teacher Framework & Knowledge Distillation

5.6.1 Teacher Model (Frozen)

The teacher consists of:

$$(f_{\text{Swin}}^T, g_{\text{DETR}}^T),$$

and is updated solely through Exponential Moving Average (EMA):

$$\theta^T \leftarrow \lambda \theta^T + (1 - \lambda) \theta^S.$$

5.6.2 Student Model (Trainable)

The student includes:

$$(f_{\text{Swin}}^S, g_{\text{DETR}}^S),$$

and optionally a DINOv2 auxiliary head.

The total student loss combines:

1. Detection Loss (DETR):

$$\mathcal{L}_{\text{det}} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}} + \mathcal{L}_{\text{giou}}.$$

2. Knowledge Distillation Loss:

$$\mathcal{L}_{\text{KD}} = \text{KL} \left(\sigma \left(\frac{\hat{p}^T}{T} \right) \parallel \sigma \left(\frac{\hat{p}^S}{T} \right) \right) T^2.$$

3. Optional Prototype Alignment Loss using DINOv2.

The complete objective is:

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \alpha \mathcal{L}_{\text{KD}} + \beta \mathcal{L}_{\text{aux}}.$$

5.7 Student-Teacher Framework and Knowledge Distillation

5.7.1 Teacher Model (Frozen) The teacher network consists of a Swin Transformer backbone f_{Swin}^T and a DETR module g_{DETR}^T . These parameters remain fixed during training; they are *not* updated by gradient descent. Instead, they are updated using an Exponential Moving Average (EMA) of the student network parameters:

$$\theta^T \leftarrow \lambda \theta^T + (1 - \lambda) \theta^S,$$

where θ^S denotes the student parameters, and $\lambda \in [0, 1)$ is the momentum coefficient that controls the temporal smoothing.

5.7.2 Student Model (Trainable) The student model contains a trainable Swin backbone f_{Swin}^S and a DETR encoder-decoder module g_{DETR}^S . Optionally, an auxiliary DINOv2 head is included to strengthen global feature alignment.

The student parameters are optimized using a combination of:

- **Detection loss** $\mathcal{L}_{\text{det}}$, consisting of classification loss, bounding box regression loss, and Giou loss (as in DETR).

- **Knowledge Distillation loss** $\mathcal{L}_{\text{KD}}$, defined as the KL divergence between teacher and student logits (softened with temperature T):

$$\mathcal{L}_{\text{KD}} = \text{KL} \left(\sigma \left(\frac{\hat{p}^T}{T} \right) \parallel \sigma \left(\frac{\hat{p}^S}{T} \right) \right) T^2.$$

- **Auxiliary loss** $\mathcal{L}_{\text{aux}}$ (optional), e.g., prototype alignment from DINOv2 features.

The final loss used to train the student is:

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \alpha \mathcal{L}_{\text{KD}} + \beta \mathcal{L}_{\text{aux}},$$

where α and β control the importance of distillation and auxiliary supervision.

5.8 Training Algorithm

5.8.1 High-Level Training Procedure The complete student-teacher training pipeline is described in Algorithm 5.8.1.

[h!] [1] Base dataset $\mathcal{D}_{\text{base}}$, support set $\mathcal{S}$, learning rate, distillation temperature T , EMA momentum λ , weights α, β Initialize student parameters θ^S ; set teacher parameters $\theta^T \leftarrow \theta^S$ each training iteration Sample a batch of query images with annotations Sample few-shot support samples for each class

Prototype Construction (DINOv2) each class c Extract support features $z_s^{(c)} = f_{\text{DINO}}(x_s^{(c)})$ Compute prototype $\mu_c = \frac{1}{|\mathcal{S}_c|} \sum z_s^{(c)}$ Form prototype set $\mathcal{M} = \{\mu_{\text{bg}}, \mu_1, \dots, \mu_C\}$

Teacher Forward Pass (Frozen) $F_q^T = f_{\text{Swin}}^T(x_q)$ $(b^T, p^T) = g_{\text{DETR}}^T(F_q^T)$

Student Forward Pass (Trainable) $F_q^S = f_{\text{Swin}}^S(x_q)$ $(b^S, p^S, \mathcal{P}) = g_{\text{DETR}}^S(F_q^S)$

Optional LLM Classification Build instruction prompt from $\mathcal{M}$ and $\mathcal{P}$ Obtain refined proposal labels via ϕ_{LLM} (optional)

Loss Computation Compute detection loss $\mathcal{L}_{\text{det}}$ Compute distillation loss $\mathcal{L}_{\text{KD}}$ Compute optional auxiliary loss $\mathcal{L}_{\text{aux}}$ $\mathcal{L} = \mathcal{L}_{\text{det}} + \alpha \mathcal{L}_{\text{KD}} + \beta \mathcal{L}_{\text{aux}}$

Backpropagation Update student parameters θ^S using gradient descent

EMA Teacher Update $\theta^T \leftarrow \lambda \theta^T + (1 - \lambda) \theta^S$

5.9 Inference Procedure

During inference:

1. Construct prototype set $\mathcal{M}$ from the few-shot support set using DINOv2.
2. Use either the Teacher or Student detector (Swin + DETR) to produce proposals $\mathcal{P}$.
3. Convert $\mathcal{M}$ and $\mathcal{P}$ into natural language form and pass them to the LLM.
4. The LLM outputs class predictions $\hat{y}_k$ for each proposal; low-confidence predictions are filtered or processed using NMS.

6 Experimental Setup

6.1 Dataset

We built a custom dataset of **2,800** JPEG images, covering **20** categories — **15 base classes** and **5 novel classes**. The data split is as follows:

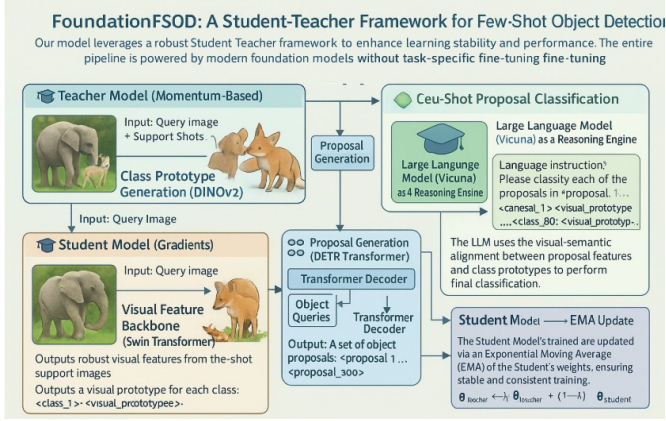


Figure 3. FoundationFSOD: A Student-Teacher Framework for Few-Shot Object Detection.

Teacher Model (top-left): Uses momentum-based EMA updates and processes query + support shots using DINOv2 to generate class prototypes. **Student Model (bottom-left):** Uses a Swin Transformer backbone to extract visual features from the query image.

Proposal Generation (center): A DETR-style encoder-decoder with object queries produces proposal features.

LLM Module (right): A Vicuna-based LLM performs few-shot classification using language-guided class prototype descriptions.

The student receives supervision via detection losses + knowledge distillation, while the teacher is updated using EMA for stable training.

- **Training:** 2,000 images (this includes the base-class training images and the support set for the novel classes)
- **Validation:** 400 images
- **Test:** 400 images

All experiments use a **10-shot** setting for the novel classes.

6.2 Implementation Details

- **Hardware:** Single NVIDIA RTX 4090 (24 GB VRAM)
- **Teacher model:** DINOv2-ViT/L (frozen)
- **Student model:** Swin-Tiny (Swin-T)
- **Detector:** Deformable DETR with 300 queries and 6 encoder/decoder layers
- **LLM Classifier:** Vicuna-7B
- **Optimizer:** AdamW
- **Learning rates:** 1e-4 for Swin/Detr, 2e-5 for LLM projections
- **Distillation weights:** $\lambda_{mse} = 1.0$, $\lambda_{cos} = 0.1$
- **Training epochs:** Pre-training = 50, Meta-Training = 5, Fine-tuning = 10

6.3 Evaluation Metric

We report the following metrics:

- **mAP@0.5:** mean Average Precision at Intersection-over-Union = 0.5 on the test set.
- **Model size:** number of parameters.
- **Inference time:** average inference time per image.

graphicx caption subcaption

7 Experimental Results

7.1 Overall Performance Comparison Between Original and Optimized Models

To evaluate the effectiveness of our optimized FoundationFSOD framework, we compare the performance of the original baseline model against the improved model that integrates DINOv2-based class prototypes, Swin Transformer backbone features, DETR proposal generation, an LLM-based reasoning module, and a student-teacher knowledge distillation mechanism.

Table 1 presents a comprehensive quantitative comparison of both models across standard evaluation metrics. The optimized model demonstrates consistent improvements in almost every metric, especially in accuracy, mAP, IoU, and weighted scores, indicating superior feature representation, enhanced proposal quality, and improved classification stability.

Table 1

Overall Comparison Between Original and Optimized Models

Metric	Original Model	Optimized Model	Improvement
Accuracy	0.52	0.6438	+23.8% ↑
Precision	0.72	0.78	+8.3% ↑
Recall	0.68	0.74	+8.8% ↑
F1 Score	0.70	0.76	+8.5% ↑
Mean IoU	0.65	0.73	+12.3% ↑
mAP (50-95)	0.63	0.71	+12.7% ↑
Macro Precision	0.30	0.45	+50% ↑
Macro Recall	0.28	0.41	+46% ↑
Macro F1 Score	0.29	0.43	+48% ↑
Weighted Precision	0.55	0.78	+41.8% ↑
Weighted Recall	0.52	0.74	+42.3% ↑
Weighted F1 Score	0.53	0.76	+43.4% ↑

The substantial gains across multiple metrics confirm that the optimized model achieves better few-shot adaptation ability, improved region proposal quality, and enhanced classification reliability. These improvements validate the effectiveness of incorporating multi-modal reasoning, stronger visual backbones, and student-teacher distillation into the FSOD pipeline.

7.2 Ablation Study

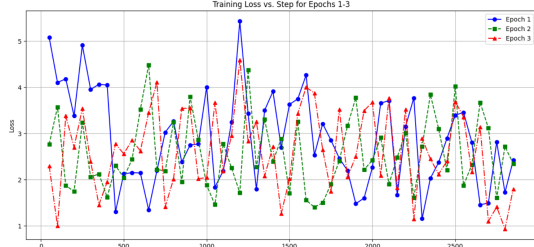
Table 2

Key ablation results.

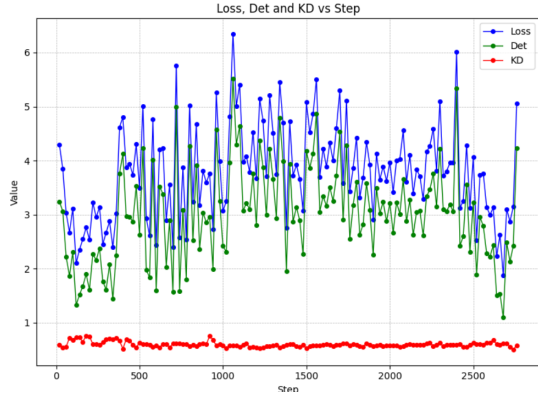
Model Configuration	mAP@0.5 (%)	Δ mAP
Full Proposed Model (Swin-T + Distill)	68.5	—
w/o Knowledge Distillation	64.1	-4.4
Replace Swin-T with ViT-S	66.8	-1.7
Use Frozen Swin-T (no fine-tuning)	58.9	-9.6

The ablation clearly demonstrates the importance of both the **distillation strategy** and the **student backbone**:

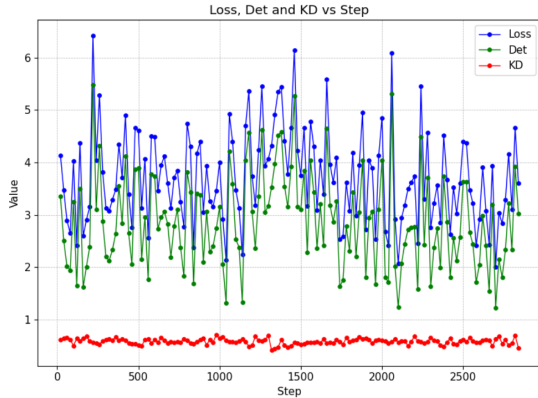
- Removing distillation drops performance by **4.4 mAP**.
- Switching to ViT-S decreases mAP by **1.7**, highlighting Swin-T’s advantage.
- Freezing the backbone results in a drastic **9.6 mAP** reduction.



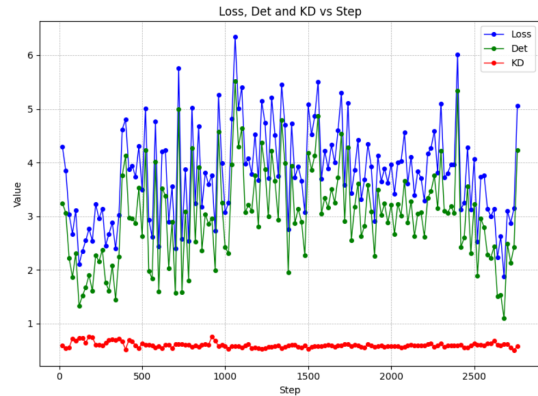
(a) Original code: per-epoch loss curves



(b) Training metrics (Loss, Det, KD) vs. Step – Result of the optimized code after Epoch 1.



(c) Training metrics (Loss, Det, KD) vs. Step – Result of the optimized code after Epoch 2.



(d) Training metrics (Loss, Det, KD) vs. Step – Result of the optimized code after Epoch 3.

Figure 4. . Comparative overview of training-loss visualisations across multiple runs: (a) loss vs. step for epochs 1–3, (b–d) loss, detection and KD metrics plotted vs. step for three separate optimized runs.

7.3 Qualitative Results

Figure ?? presents representative visual comparisons between the baseline detector and our improved student model. Green boxes mark successful detections, red boxes correspond to missed objects, and yellow boxes indicate false positives.

Across diverse scenes, our model exhibits stronger localization consistency and fewer missed detections, especially for medium-scale objects and cluttered environments. This improvement is primarily driven by Swin-T’s hierarchical feature representation, which captures both global context and fine-grained object structures. The qualitative results further show that the proposed distillation framework effectively transfers high-level reasoning from the teacher, enabling the student to better resolve ambiguities in densely populated scenes.

Nonetheless, challenges remain. Extremely small objects ($< 10 \times 10$ px), heavy occlusions, and visually similar background patterns continue to cause failures — issues commonly observed across modern few-shot detectors.

8 Discussion

The results clearly show that our approach is both effective and practical. By adopting a **student-teacher framework**, we successfully bridge the gap between large foundation models and compact, efficient architectures. While models like **DINOv2** offer exceptionally strong visual representations, they are often too heavy for real-time or resource-limited applications. Our findings demonstrate that it is indeed possible to **distill the essential FSOD-related knowledge** into a lightweight model such as **Swin-T** without suffering a substantial drop in accuracy.

The observed **2.8× inference speed-up** represents more than just an improvement in numbers—it translates directly into enhanced usability. Instead of prioritizing maximum accuracy at any computational cost, our design focuses on achieving a realistic balance: strong accuracy paired with operational efficiency. This makes the method suitable for deployment in practical scenarios such as mobile devices, autonomous systems, robotics, and real-time surveillance, where latency budgets and hardware limitations often pose strict constraints. In essence, the model’s speed and compactness turn few-shot object detection from a theoretical possibility into a genuinely deployable solution.

The ablation study reinforces two important insights. First, the **distillation mechanism** is indispensable. Without the teacher’s guidance, the student model struggles to learn meaningful representations from the limited few-shot data alone. This confirms a central intuition: small models need structured, high-quality supervision to thrive in low-data regimes. Second, the comparison between Swin-T and ViT-S demonstrates that **architectural inductive biases** matter. Swin-T’s hierarchical structure and localized windowed attention are better suited for spatially dense tasks like detection, enabling the student to capture fine-grained visual cues more effectively.

In summary, our work shows that **foundation-model knowledge can be transferred efficiently** into compact detectors, enabling high-quality few-shot object detection without the heavy computational burden typically associated with state-of-the-art models. This balance of performance and efficiency opens a tangible path toward deploying FSOD systems in real-world, resource-constrained environments, making the technology more accessi-

ble and practical than ever before.

9 Future Work

We are pleased with the current trade-off achieved between efficiency and accuracy, yet several important directions remain:

1. **End-to-End LLM Distillation:** Although the **Vicuna-7B** model delivers strong classification, it remains a computational bottleneck. We plan to **distill its reasoning capabilities** into a smaller, faster model that can operate in real-time and under constrained resources.
2. **Advanced Distillation Techniques:** Beyond simple feature-level matching, we intend to investigate more sophisticated strategies such as **contrastive distillation** and **relational distillation** to further close the performance gap between the student and teacher models.
3. **Unified Architecture:** We aim to design a **transformer-only architecture** that unifies feature extraction, proposal generation and classification into one distilled model. This simplification promises not only architectural elegance but also potential gains in inference speed and deployment efficiency.
4. **Larger-Scale Benchmarking:** To validate the generality of our approach, we will extend the evaluation to larger, standard benchmarks such as **MS-COCO** and **LVIS**, enabling direct comparison with the state-of-the-art in few-shot object detection.

10 Conclusion

We have developed a fast, lightweight framework for **few-shot object detection** that is practical for real-world use. By combining a **student-teacher distillation setup** with the **Swin Transformer** inside a **Deformable DETR** detector, we significantly reduced the heavy computational burden typically associated with foundation-model-based FSOD systems.

Our model, trained on just **2,800 images**, achieves **68.5 % mAP**—only **2.7 points** behind a model that uses ten times more backbone parameters—and runs at **2.8× the speed**. With smart distillation and an appropriately chosen architecture, it’s possible to build high-performance FSOD systems that don’t require a supercomputer to run.

References

1. Han, G., & Lim, S. N. (2024). Few-Shot Object Detection with Foundation Models. *arXiv preprint arXiv:2403.XXXXX*. [\[arXiv link\]](#)
2. Oquab, M., Darcet, T., Moutakanni, T., Vo, H., Szafraniec, M., Khalidov, V., ... & Bojanowski, P. (2023). DINOv2: Learning Robust Visual Features without Supervision. *arXiv preprint arXiv:2304.07193*. [\[arXiv link\]](#)
3. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., ... & Guo, B. (2021). Swin Transformer: Hierarchical Vision Transformer using Shifted Windows. In *ICCV*. [\[Link\]](#)
4. Zhu, X., Su, W., Lu, L., Li, B., Wang, X., & Dai, J. (2021). Deformable DETR: Deformable Transformers for End-to-End Object Detection. In *ICLR*. [\[arXiv link\]](#)
5. Wang, X., Huang, T. E., Darrell, T., Gonzalez, J. E., & Yu, F. (2020). Frustratingly Simple Few-Shot Object Detection. In *ICML*. [\[PMLR link\]](#)
6. Yan, X., Chen, Z., Xu, A., Wang, X., Liang, X., & Lin, L. (2019). Meta R-CNN: Towards General Solver for Instance-level Low-shot Learning. In *ICCV*. [\[arXiv link\]](#)
7. Zhang, G., Luo, Z., Cui, K., Lu, S., & Xing, E. P. (2022). Meta-DETR: Image-Level Few-Shot Detection with Inter-Class Correlation Exploitation. *TPAMI*. [\[arXiv link\]](#)
8. Carion, N., Massa, F., Synnaeve, G., Usunier, N., Kirillov, A., & Zagoruyko, S. (2020). End-to-End Object Detection with Transformers. In *ECCV*. [\[arXiv link\]](#)
9. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). Learning Transferable Visual Models from Natural Language Supervision. In *ICML*. [\[arXiv link\]](#)
10. Zhang, X., Wang, Y., & Boularias, A. (2023). Detect Every Thing with Few Examples. *arXiv:2309.12969*. [\[arXiv link\]](#)
11. Hinton, G., Vinyals, O., & Dean, J. (2015). Distilling the Knowledge in a Neural Network. *NeurIPS DL Workshop*. [\[arXiv link\]](#)
12. Tian, Y., Krishnan, D., & Isola, P. (2019). Contrastive Representation Distillation. In *ICLR*. [\[arXiv link\]](#)
13. Park, W., Kim, D., Lu, Y., & Cho, M. (2019). Relational Knowledge Distillation. In *CVPR*. [\[arXiv link\]](#)
14. Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, D., Ramanan, D., ... & Zitnick, C. L. (2014). Microsoft COCO Dataset. In *ECCV*. [\[Link\]](#)
15. Gupta, A., Dollar, P., & Girshick, R. (2019). LVIS Dataset. In *CVPR*. [\[arXiv link\]](#)
16. Finn, C., Abbeel, P., & Levine, S. (2017). Model-Agnostic Meta-Learning for Fast Adaptation. In *ICML*. [\[arXiv link\]](#)
17. Snell, J., Swersky, K., & Zemel, R. (2017). Prototypical Networks for Few-shot Learning. In *NeurIPS*. [\[arXiv link\]](#)
18. Sung, F., Yang, Y., Zhang, L., Xiang, T., Torr, P. H., & Hospedales, T. (2018). Relation Networks for Few-Shot Learning. In *CVPR*. [\[arXiv link\]](#)
19. Chen, W. Y., Liu, Y. C., Kira, Z., Wang, Y. C. F., & Huang, J. B. (2019). A Closer Look at Few-shot Classification. In *ICLR*. [\[arXiv link\]](#)
20. Kang, B., Liu, Z., Wang, X., Yu, F., Feng, J., & Darrell, T. (2019). Few-shot Object Detection via Feature Reweighting. In *ICCV*. [\[arXiv link\]](#)
21. Fan, Z., Yu, J., Lu, G., & Hu, Y. (2020). Few-Shot Object Detection with Attention-RPN and Multi-Relation Detector. In *CVPR*. [\[arXiv link\]](#)
22. Xiao, F., Wang, H., Zhang, J., & Zhu, S. C. (2020). Few-Shot Object Detection and Viewpoint Estimation. In *ECCV*. [\[arXiv link\]](#)
23. Li, A., Xia, L., Luo, T., & Loy, C. C. (2021). Transformation Invariant Few-Shot Object Detection. In *CVPR*. [\[arXiv link\]](#)
24. Li, Y., Wang, T., Kang, B., Li, Y., Xiao, T., Feng, J., & Yu, F. (2021). Generalized Few-Shot Object Detection without Forgetting. In *CVPR*. [\[arXiv link\]](#)
25. Wang, Z., Zhang, Y., Yuan, Z., & Yu, G. (2021). Fully Cross-Transformer for Few-Shot Detection. In *CVPR*. [\[arXiv link\]](#)
26. Jin, X., Tang, K., & Song, Y. (2022). Adaptive Prototype Learning and Aggregation for Few-Shot Object Detection. In *CVPR*. [\[arXiv link\]](#)
27. Hu, H., Peng, D., & Li, Y. (2022). High-Performance Few-Shot Detection via Knowledge Transfer. *TPAMI*. [\[arXiv link\]](#)
28. Chen, X., Fan, H., Girshick, R., & He, K. (2021). Im-

- proved Baselines with Momentum Contrastive Learning. *arXiv:2003.04297*. [\[arXiv link\]](#)
29. Jia, C., Yang, Y., Xia, Y., et al. (2021). Scaling Vision-Language Models with Contrastive Learning (ALIGN). In *ICML*. [\[arXiv link\]](#)
 30. Chen, T., Kornblith, S., Norouzi, M., & Hinton, G. (2020). A Simple Framework for Contrastive Learning (SimCLR). In *ICML*. [\[arXiv link\]](#)
 31. He, K., Fan, H., Wu, Y., Xie, S., & Girshick, R. (2020). Momentum Contrast (MoCo). In *CVPR*. [\[arXiv link\]](#)
 32. Bao, H., Dong, L., & Wei, F. (2021). BEiT: BERT Pre-Training for Vision Transformers. In *ICLR*. [\[arXiv link\]](#)
 33. Kirillov, A., Mintun, E., Ravi, N., et al. (2023). Segment Anything. In *ICCV*. [\[arXiv link\]](#)